

ClaudeOS

AI Operating System

Agent Registry Documentation • Version 1.0 • May 2026

ClaudeOS is an AI Operating System built for automation-first power users. It unifies 12 specialised agents, a persistent memory engine, workflow automation, client namespace isolation, and a full authentication layer into a single authenticated control centre. This document describes each agent — what it does, how it works, and a practical example of its output.

12 Agents	8 Categories	claude-sonnet-4-6 Model	SQLite + ChromaDB Memory
---------------------------------	------------------------------------	---	--

Table of Contents

01	Morning Briefing Agent	OPS	agent-briefing-001
02	Client Manager Agent	OPS	agent-client-manager-001
03	Communications Agent	COMMS	agent-comms-001
04	Research Agent	RESEARCH	agent-research-001
05	Analysis Agent	ANALYSIS	agent-analysis-001
06	Writing Agent	CONTENT	agent-writing-001
07	QA Agent	ENGINEERING	agent-qa-001
08	RECI Transport Ops Agent	DOMAIN	agent-transport-ops-001
09	Scheduling Agent	OPS	agent-scheduling-001
10	Memory Curator Agent	SYSTEM	agent-memory-curator-001
11	Meta Orchestrator Agent	SYSTEM	agent-meta-001
12	Workflow Builder Agent	SYSTEM	agent-workflow-builder-001

01 Morning Briefing Agent

agent-briefing-001

OPS

The Morning Briefing Agent synthesises overnight context into a structured, actionable daily brief every weekday. It scans memory for urgent items, project updates, client activities, and scheduled workflows, then delivers a concise five-section report: Priority Actions (top 3 items), Project Status (one-line per active project), Client Updates, Scheduled Work, and Notes. It operates on WAT (UTC+1) timezone and pulls exclusively from live memory context — nothing is invented. Urgent or reminder-tagged items are always surfaced first. Designed for Rigwe's automation-first workflow, it eliminates the daily overhead of manually checking project states across multiple client namespaces.

EXAMPLE

Prompt: 'Generate morning briefing for Friday 22 May 2026.'

Output: Priority action — 'RECI Dashboard: await Chinedu reply, June 15 deadline is 24 days out.' Project status table across 4 active projects. Client update noting follow-up email sent to chinedu@recitransport.com.

02 Client Manager Agent

agent-client-manager-001

OPS

The Client Manager Agent generates structured status cards for every active client namespace. Each card contains: client name and type, active projects with on-track / at-risk / blocked status, next milestone with date, current blockers, actions required from Rigwe, and actions required from the client. It draws data from memory context and flags any missing information as [UNKNOWN] rather than inventing details — ensuring cards are always accurate. This agent is the primary tool for weekly client reviews, keeping deliverables visible and accountability clear across all three client namespaces: Ivycandy Hair, RECI Transport, and Website Portal.

EXAMPLE

Prompt: 'Generate status cards for all 3 clients.'

Output: Three markdown cards — Ivycandy Hair (Design System: active, no deadline set), RECI Transport (Client Dashboard: blocked, June 15 target), Website Portal (Client Portal: active, role matrix undocumented).

03

Communications Agent

agent-comms-001

COMMS

The Communications Agent drafts all client-facing communications — emails, WhatsApp messages, proposals, and follow-ups. It tone-matches per client by reading their profile from memory context. The default register is professional but warm, aligned with Nigeria B2B communication standards. It never invents facts or figures; unknowns are clearly marked as [FILL IN] placeholders so the user can complete before sending. Email output always includes a subject line and signature block. WhatsApp messages are shorter and conversational with no formal headers. All output is ready-to-send with no internal ClaudeOS references included.

EXAMPLE

Prompt: 'Send a follow-up email to RECI Transport about the dashboard.'

Output: Subject: 'Client Dashboard Project — Quick Alignment on Data, KPIs & Go-Live Date.' Professional email body requesting confirmation on data source, KPI scope, and June 15 deadline. Signed by Romanus Igwe, Operations.

04

Research Agent

agent-research-001

RESEARCH

The Research Agent conducts deep, structured research on any topic and synthesises findings into a validated report. For every request it identifies what is known versus what requires verification, structures findings into Overview, Key Facts, Key Players, Implications, and Sources sections, and distinguishes high-confidence facts from uncertain claims. Each claim is labelled by source type: memory, known fact, or inference. Gaps — things that could not be verified — are explicitly flagged. Every report ends with three recommended next actions. It operates at 8,192 max tokens, making it the highest-capacity agent in the registry.

EXAMPLE

Prompt: 'What is the best data source for the RECI dashboard: SQLite or Supabase?'

Output: Full comparison table across 7 factors (offline access, multi-user, Nigeria network, cost, complexity).

Recommendation: start with SQLite, migrate to Supabase when team size confirmed. Three next actions listed.

05

Analysis Agent

agent-analysis-001

ANALYSIS

The Analysis Agent processes raw data and produces structured analytical reports. Every report follows a fixed six-part structure: Executive Summary (2-3 sentences), Key Findings (bullet list), Anomalies (unusual or concerning patterns), Trends (patterns over time), Recommendations (3-5 concrete next actions ranked by impact), and a Confidence rating of HIGH / MEDIUM / LOW with justification. It never speculates beyond the provided data, flags data quality issues explicitly, and uses exact numbers without rounding unless noted. Temperature is set to 0.2 — the most deterministic of the non-system agents — ensuring consistent, repeatable analytical outputs.

EXAMPLE

Prompt: 'Analyse RECI Transport booking data for Q1 2026.'

Output: Executive summary of volume trends, key finding that Lagos-Abuja route accounts for 42% of bookings, anomaly flagging a 30% drop in week 8, trend showing Friday peak bookings, and three recommendations including dynamic pricing for peak slots.

06

Writing Agent

agent-writing-001

CONTENT

The Writing Agent produces polished written content matched to the voice and tone of the active namespace. It strictly avoids mixing client voices or styles — Ivycandy Hair copy reads differently from RECI Transport reports. Facts are never invented; unknowns are marked as [FILL IN]. Client-facing content uses professional, jargon-free British English (Nigeria business standard). Internal content is direct and terse. It handles drafts, reports, proposals, landing page copy, and formatted documentation. Operating at temperature 0.7 — the highest among content agents — it produces natural, varied prose rather than mechanical output.

EXAMPLE

Prompt: 'Write a project proposal for the Ivycandy Hair Design System.'

Output: Professional proposal with executive summary, scope of work (brand tokens, component library, competitive analysis of 5 hair stores), deliverables list, timeline, and investment section with [FILL IN] placeholder for pricing.

The QA Agent reviews code for bugs, security vulnerabilities, and quality issues. It checks against OWASP Top 10 (SQL injection, XSS, IDOR, and more), Windows compatibility (UTF-8 encoding, PowerShell syntax, no Unix-only commands), hardcoded secrets, brand compliance (#407E3C green + white for UI code), and common bugs including off-by-one errors, null handling, and exception swallowing. Output is a structured markdown table with columns: Severity, File, Line, Issue, Fix. Severity levels are CRITICAL (security / data loss), HIGH (correctness bugs), MED (quality issues), LOW (optional improvements), and INFO.

EXAMPLE

Prompt: 'Review core/auth.py for security issues.'

Output: Table finding CRITICAL — hardcoded fallback secret key on line 47; HIGH — bare except swallowing auth exceptions on line 112; MED — missing LOWER() on username lookup causing case-sensitive login failure. Each finding includes the exact fix.

The RECI Transport Ops Agent is the only namespace-locked agent in ClaudeOS — it operates exclusively on reci-transport data and refuses cross-namespace access. It is the domain expert for Nigerian transport and fleet operations. Every report covers Fleet Utilisation (% vehicles booked vs available), Booking Trends (volume, peak times, popular routes), Revenue Snapshot, Anomalies, and Action Items. It understands Nigerian context: prices in NGN, major hubs (Lagos, Abuja, Port Harcourt), peak seasons (December, Easter, Eid), and fuel cost as a key variable. Output uses tables for all numbers.

EXAMPLE

Prompt: 'Generate fleet utilisation report for May 2026.'

Output: Fleet at 73% utilisation, Lagos-Abuja at capacity Friday-Sunday, revenue NGN 4.2M for the month, anomaly flagging two vehicles with zero bookings for 3 consecutive weeks (maintenance risk), action: reassign idle vehicles to Ibadan route where demand exceeds supply.

The Scheduling Agent converts natural language scheduling requests into structured memory reminder entries with ISO 8601 datetimes and automatic expiry. It always converts relative times ('next Friday', 'in two weeks') to absolute UTC datetimes, uses WAT (UTC+1) for Nigeria-based tasks, and sets expiry one hour after the scheduled time so the reminder remains visible until actioned. Output is pure JSON — no explanation — containing the action, when, namespace, full memory entry definition, and a plain-English confirmation string. If the request is ambiguous, it makes the best interpretation and notes the ambiguity.

EXAMPLE

Prompt: 'Remind me to chase Chinedu on Friday if no reply.'

Output JSON: action — 'Follow up with Chinedu re: RECI dashboard data source', when — '2026-05-29T08:00:00+01:00', namespace — 'reci-transport', expires_at — '2026-05-29T09:00:00+01:00', confirmation — 'Scheduled chase reminder for Friday 29 May at 08:00 WAT.'

The Memory Curator Agent maintains quality and hygiene across the ClaudeOS memory store. Given a batch of memory entries, it identifies duplicates (same key or >90% similar value), contradictions (same key, conflicting values), and stale entries (outdated by age and category). It recommends merges (keeping the newest, highest-confidence entry), expirations, and entries to keep unchanged. Output is structured JSON with merge, expire, keep, and summary keys. It is the most conservative agent in the system: when in doubt it keeps entries, and it never expires permanent categories (fact, decision, preference) unless confidence falls below 0.3.

EXAMPLE

Prompt: 'Curate global namespace memory (6 entries).'

Output JSON: expire — ['test.handover' (superseded by handover.final)], keep — 5 entries including user profile, build skill, and handover marker. Summary: 'No duplicates or contradictions found. One redundant draft entry recommended for expiration.'

11

Meta Orchestrator Agent

agent-meta-001

SYSTEM

The Meta Orchestrator Agent is the coordination brain of ClaudeOS. It decomposes complex, multi-step goals into ordered agent invocation chains of up to five steps. For each step it specifies: which agent to use, the exact ready-to-execute prompt, the namespace, dependencies on prior steps, output variable names for chaining, and a rationale. Output is pure JSON — a complete execution plan. It does not execute the plan itself; it produces the blueprint that drives multi-agent workflows. It knows all 11 registered agents and their specialisations, and enforces the rule that transport operations always route to transport-ops-agent.

EXAMPLE

Prompt: 'Generate status cards for all 3 clients.'

Output JSON plan: Step 1-3 — run client-manager-agent on each namespace in parallel (ivycandy-hair, reci-transport, website-portal). Step 4 — pass all three outputs to writing-agent to format into polished cards. Estimated 4 steps, 4,800 tokens.

12

Workflow Builder Agent

agent-workflow-builder-001

SYSTEM

The Workflow Builder Agent converts plain-English automation descriptions into complete, valid ClaudeOS workflow YAML definitions. Given a description like 'run a morning briefing every weekday at 7am', it outputs a fully structured YAML with name, trigger type (cron, manual, or event), cron expression in WAT timezone, namespace, and ordered steps — each specifying the agent, input parameters, and output variable for chaining. Step IDs use snake_case. It validates agent names against the registered registry and uses {{variable}} syntax for passing outputs between steps. Output is YAML only — no explanation or commentary.

EXAMPLE

Prompt: 'Build a workflow that runs the briefing agent every weekday morning.'

*Output YAML: name — daily-morning-briefing, trigger — cron '0 6 * * 1-5' (07:00 WAT), namespace — global, step — briefing-agent with prompt 'Generate morning briefing for today', output_key — briefing_output.*

ClaudeOS v1.0

12 agents • Persistent memory • Role-based auth • Multi-client namespaces